BriefPilot — 30-Day MVP Execution Plan

Version 1.0 · July 2026 · Owner: solo full-stack engineer, AI-assisted development · Target: demo-ready, portfolio-quality MVP in 30 calendar days

---

Executive Summary

BriefPilot's MVP is a web application that lets a user upload a German official letter (photo or PDF) and receive: structured extraction (sender, document type, deadlines, required actions, legal references), a plain-language English explanation grounded in the document, an action checklist, and — the signature trust feature — **every extracted deadline highlighted directly in the original document image**.

The plan runs four one-week sprints across 30 calendar days (24 working days at 6–8 h/day, weekends mostly reserved for buffer and content work). The critical path is: **OCR with positional data → classification → schema-validated extraction → grounded explanation → highlight overlay UI**. Everything else (auth, polish, docs) hangs off that spine.

Two deliberate scope decisions, made now so they don't get relitigated mid-sprint:

1. **Reply drafting and multi-letter case management are OUT of the MVP.** They are the right 90-day features, but each adds a week of work and neither is needed to demonstrate the core engineering value. A shipped, tested, well-documented extraction pipeline beats a half-shipped feature buffet in every recruiter conversation.
2. **The evaluation suite is IN the MVP and is treated as a feature, not a chore.** A repo that publishes per-document-type extraction accuracy on a golden set of real letters is the single strongest engineering-maturity signal this project can send. It gets its own epic, its own sprint time, and a section in the README.

A note on process: classic Scrum ceremony (story points, velocity tracking, formal retros) is theater for a solo developer. This plan keeps what earns its cost — sprint goals, acceptance criteria, a daily checklist, a risk register, demo checkpoints — and drops the rest. Estimates are in **hours**, because that's the unit you actually plan a solo day in. Where story points appear, they're a coarse S/M/L tag for board readability only.

Phase 1 — MVP Definition

In scope (must exist at launch)

#	Feature	Why essential	User value	Complexity	Risk	Dep on
F1	Upload (photo/PDF,	Entry point;	Zero-	Low	Low	—

#	Feature	Why essential	User value	Complexity	Risk	Dep on
	multi-page, drag-drop + mobile camera)	nothing works without it	friction start			
F2	OCR with word-level bounding boxes	Foundation for extraction AND highlight overlay	Accurate reading of real scans	Medium	**High** (quality on phone photos)	F1
F3	Document-type classification	Each type needs its own extraction schema; also powers UX copy	Correct, type-specific handling	Medium	Medium	F2
F4	Structured extraction (schema-validated JSON): sender, recipient, doc type, dates, deadlines, amounts, required actions, legal references, contact info	The core product; the difference between a wrapper and a pipeline	"What does this letter want from me?"	High	**High** (hallucination)	F2, F
F5	Programmatic validators (dates parse; deadlines ≥ letter date; cited § exists in reference list; amounts are numeric)	Converts LLM output into trustworthy data; kills the worst hallucinations deterministically	Trust	Medium	Low	F4
F6	Plain-language explanation (DE→EN), grounded in extracted fields + document text only	The user-facing payoff	"Now I understand"	Medium	Medium	F4
F7	Action checklist with deadlines	Turns understanding into action	"What do I do, by when?"	Low	Low	F4
F8	**Source highlighting**: deadlines and key	The trust differentiator;	Verifiable output	Medium	Medium	F2, F

#	Feature	Why essential	User value	Complexity	Risk	Dep on
	fields highlighted in the original scan	proves the AI isn't inventing				
F9	Per-field confidence scores, surfaced in UI; low-confidence fields visually flagged	Honest AI UX; recruiter catnip	Knows what it doesn't know	Medium	Medium	F4
F10	Legal-term glossary popovers (curated ~50-term Amtsdeutsch glossary)	Cheap, high-perceived-value; grounds explanations	Learns the vocabulary	Low	Low	F6
F11	Results page UX: summary card, checklist, document viewer with overlay, disclaimer	Where all value is consumed	Everything	Medium	Low	F6–F
F12	Evaluation suite: 30+ golden letters, automated scoring per field per doc type, published results	Quality moat + the portfolio centerpiece	Reliability	Medium	Low	F4, F
F13	GDPR baseline: EU hosting, no-training API config, delete-my-data, no account required for first use, privacy page	Legal necessity + trust marketing	Safety	Low–Med	Medium	Depl
F14	Deployment (EU region) + landing page + README/architecture docs + demo video	It doesn't exist until it's live and documented	Access	Low	Low	All

Explicitly OUT (post-MVP backlog, in priority order)

1. Reply drafting with mandatory human-edit step (first post-MVP feature; the RDG legal-line review happens before this ships)

2. Case threads (link related letters into one case) + deadline reminders via email/calendar
3. Accounts & saved history (MVP is session-based; a share/export link covers the demo need)
4. Additional output languages (architecture keeps language as a parameter so this is config, not code)
5. Widerspruch-specific guided flows
6. Native mobile apps (MVP is a responsive PWA)
7. B2B white-label, family sharing, any DACH expansion work

Rule for the month: any new idea goes to this list, not into a sprint. Scope creep is Risk #1 in the register for a reason.

Phase 2 — Epics

Epic	Purpose
E1 Foundation	Repo, CI, environments, stack scaffolding, EU deployment target from day 1
E2 Upload & Ingestion	File intake, image preprocessing, multi-page handling
E3 OCR Pipeline	Text + word-level coordinates from photos and PDFs; quality fallback strategy
E4 Classification	Document-type detection routing to per-type schemas
E5 Structured Extraction	LLM extraction with strict JSON schemas + validator layer + confidence
E6 Explanation Engine	Grounded plain-language explanation, checklist generation, glossary
E7 Frontend Experience	Upload flow, processing states, results page, highlight overlay viewer
E8 Evaluation & Testing	Golden dataset, automated eval harness, unit/integration tests, published metrics
E9 Privacy & Deployment	GDPR baseline, EU hosting, monitoring, error tracking
E10 Documentation & Portfolio	README, architecture diagram, failure analysis, demo video, screenshots, LinkedIn cadence

Authentication is deliberately **not** an epic: no accounts in MVP. This removes ~2 days of work and an entire GDPR surface.

Phase 3 — User Stories

Format: story → acceptance criteria (AC) → priority (P0 must / P1 should) → estimate (hours, with S/M/L tag) → dependencies. Definition of Done for **every** story: code merged to main via PR, tests written and passing in CI, no console errors, works on mobile viewport, documented if it changes architecture.

E1 Foundation

- **S1.1** As the developer, I want a scaffolded monorepo (Next.js frontend + FastAPI backend) with CI, linting, and env management, so every later story ships through the same pipeline. AC: `main` deploys automatically to a live EU-region URL; CI runs lint + tests on every PR; secrets never in repo. P0 · 6h (M) · —
- **S1.2** As the developer, I want a deployed "hello pipeline" walking skeleton (upload → dummy response → render), so the full request path exists before any real logic. AC: a file uploaded on the live URL returns and displays a stubbed result end-to-end. P0 · 4h (S) · S1.1

E2 Upload & Ingestion

- **S2.1** As a new immigrant, I want to upload a photo or PDF of my letter (up to 10 pages, 20 MB), so I can get it analyzed without retyping anything. AC: drag-drop and mobile camera capture both work; PDF pages split to images; unsupported types rejected with a clear message; upload progress shown. P0 · 6h (M) · S1.2
- **S2.2** As a user with a crooked phone photo, I want automatic image cleanup (deskew, contrast, downscale), so OCR quality doesn't depend on my photography. AC: preprocessing measurably improves OCR confidence on ⩾3 test photos; originals preserved for display. P0 · 5h (M) · S2.1

E3 OCR Pipeline

- **S3.1** As the system, I need OCR returning full text **and word-level bounding boxes**, so extraction and source highlighting share one data source. AC: JSON output of `{text, page, bbox, confidence}` per word; works on scanned PDF and phone photo fixtures; per-page confidence computed. P0 · 8h (L) · S2.2
- **S3.2** As the system, I need an OCR quality gate, so garbage input fails loudly instead of producing confident nonsense downstream. AC: below-threshold pages trigger a user-facing "retake the photo" prompt with tips; threshold tuned against fixtures. P0 · 3h (S) · S3.1

E4 Classification

- **S4.1** As the system, I need each document classified into one of ~8 launch types (Finanzamt Bescheid, Ausländerbehörde, Krankenkasse, Bußgeldbescheid, Rundfunkbeitrag, Jobcenter, rental/utility, other), so the right extraction schema is

applied. AC: LLM classifier with few-shot examples; ⩾90% accuracy on golden set; "other" routes to a generic schema, never an error. P0 · 5h (M) · S3.1

E5 Structured Extraction

- **S5.1** As a user, I want the letter's key facts extracted into structured fields, so nothing important hides in the wall of text. AC: per-type Pydantic schemas; LLM constrained to JSON; every field carries `{value, confidence, source_span}` linking back to OCR words; missing fields returned as null, never guessed. P0 · 10h (L) · S4.1
- **S5.2** As the system, I need deterministic validators on extraction output, so impossible values never reach the user. AC: dates parse and are calendar-valid; deadlines ⩾ letter date (violation → flag, not silent fix); cited legal references checked against a curated § list; amounts numeric; validator failures downgrade field confidence and flag for review. P0 · 6h (M) · S5.1
- **S5.3** As a user, I want to see how confident BriefPilot is per field, so I know what to double-check. AC: three visual confidence tiers; low-confidence fields carry a "verify this against the highlighted source" prompt. P0 · 3h (S) · S5.2

E6 Explanation Engine

- **S6.1** As a user, I want a plain-English explanation of what the letter says and why I received it, so I understand without a dictionary. AC: explanation generated from document text + extracted fields only (prompt forbids outside knowledge); ⩽200 words; B1-English readability; every factual claim traceable to the document; non-advice disclaimer rendered. P0 · 6h (M) · S5.1
- **S6.2** As a user, I want a prioritized action checklist with deadlines, so I know exactly what to do next. AC: actions derived from extracted `required_actions` + `deadlines`; sorted by date; urgent (<14 days) visually flagged; empty state handled ("no action required — here's why"). P0 · 4h (S) · S6.1
- **S6.3** As a user, I want tricky Amtsdeutsch terms explained inline, so I learn the vocabulary as I go. AC: curated 50-term glossary (JSON, versioned); terms in explanation get popover definitions; glossary is the only source for term definitions. P1 · 4h (S) · S6.1

E7 Frontend Experience

- **S7.1** As a user, I want a results page with summary card, checklist, and explanation, so everything I need is on one screen. AC: responsive; loading/processing states with honest progress steps; error states designed, not default. P0 · 8h (L) · S6.2
- **S7.2** As a skeptical user, I want extracted deadlines and amounts highlighted in my original document image, so I can verify the AI isn't inventing. AC: click a field → viewer scrolls to page and draws bbox overlay; works multi-page; overlay coordinates correct after image scaling; this feature is in the demo video. P0 · 8h (L) · S5.1, S7.1
- **S7.3** As a privacy-conscious user, I want to analyze a letter without creating an account and delete my data with one click, so trying the product is safe. AC: no auth

wall; "Delete everything" removes files + records and confirms; auto-purge after 24h documented and implemented. P0 · 4h (S) · S7.1

E8 Evaluation & Testing

- **S8.1** As the developer, I want a golden dataset of ≥30 anonymized real letters with hand-labeled expected extractions, so quality is measured, not vibed. AC: sourced from own mail, friends, public samples; PII scrubbed; labels stored as fixture JSON; ≥3 examples per launch doc type. P0 · 8h (L) spread across sprints · —
- **S8.2** As the developer, I want an automated eval harness scoring extraction accuracy per field per doc type, so any prompt/model change is regression-tested. AC: one command runs full eval, outputs a scorecard (per-field precision/recall, deadline-date exact-match rate); scorecard published in README; CI runs eval on a fast subset. P0 · 6h (M) · S8.1, S5.2
- **S8.3** As the developer, I want unit + integration tests on the pipeline seams, so refactors are safe. AC: validators 100% unit-tested; OCR→extraction integration test on 3 fixtures; upload E2E happy path via Playwright. P0 · 6h (M) · rolling

E9 Privacy & Deployment

- **S9.1** As the operator, I want production deployment in an EU region with error tracking and basic analytics, so failures are visible before users report them. AC: EU-region hosting; Sentry (EU) wired; uptime check; API keys configured with no-training/data-retention-off options where offered. P0 · 4h (S) · S1.1
- **S9.2** As a user, I want a clear privacy page stating storage location, retention, deletion, and that documents never train models, so I can trust the upload. AC: plain-language privacy page; linked from upload screen; retention claims match actual implementation. P0 · 3h (S) · S7.3

E10 Documentation & Portfolio

- **S10.1** As a hiring manager skimming the repo, I want a README that explains the problem, architecture, eval results, and known failure modes in 3 minutes, so I can judge maturity fast. AC: architecture diagram; eval scorecard table; **honest failure-analysis section** ("the 10–15% we get wrong and why"); competitor acknowledgment paragraph; setup instructions verified on a clean machine. P0 · 6h (M) · S8.2
- **S10.2** As the developer, I want a 3-minute demo video and screenshot set, so the project is consumable without running it. AC: video shows upload → results → source-highlight verification → deletion; hosted and linked from README and LinkedIn. P0 · 4h (S) · S7.2

Total core estimate: ~130–140 h against ~150–170 h available. The ~20% buffer is not slack — it will be eaten by OCR edge cases and prompt iteration. Plan on it.

Phase 4 & Timeline — Sprint Plans

Gantt overview (30 calendar days, Mon Day-1 start)

Workstream	D1-2	D3-5	D6-7	D8-10	D11-12	D13-14	D15-17	D18-19	D20-21	D22-24	D25-26
E1 Foundation	■										
E2 Upload		■									
E3 OCR		■	■								
E4 Classification				■							
E5 Extraction+Validators				■	■						
E6 Explanation						■	■				
E7 Frontend/Overlay							■	■	■		
E8 Eval/Testing		▒	▒	▒	▒	■	▒	▒	■	■	
E9 Privacy/Deploy	▒									■	■
E10 Docs/Portfolio	▒		▒			▒					■

(■ = focus work, ▒ = background/rolling)

Sprint 1 (Days 1–7): "A letter goes in, real text comes out"

- **Business objective:** de-risk the single scariest dependency — OCR quality on real-world photos — in week one.
- **Engineering objective:** live walking skeleton + upload + preprocessing + OCR with bounding boxes + quality gate.
- **Stories:** S1.1, S1.2, S2.1, S2.2, S3.1, S3.2; start S8.1 (collect first 10 golden letters).
- **Expected demo (Day 7):** on the live URL, photograph a real Finanzamt letter with a phone; see extracted raw text with per-word confidence rendered over the image.
- **Risks:** OCR provider choice churn (mitigate: timebox a Day-3 bake-off, decide, commit); scanned-PDF vs photo divergence (mitigate: fixtures for both from day one).
- **DoD:** all S1–S3 ACs met; CI green; skeleton deployed; 10 golden letters collected.
- **Review checklist:** upload works on a real phone; OCR JSON includes bboxes; quality gate triggers on a deliberately bad photo.

- **Retro questions:** Was the OCR bake-off decisive? Is the 6–8h/day pace real? What surprised me about real letters?

Sprint 2 (Days 8–14): "The pipeline understands the letter"

- **Business objective:** produce trustworthy structured data — the product's core claim.
- **Engineering objective:** classification, schema-constrained extraction, validator layer, confidence scoring, eval harness v1.
- **Stories:** S4.1, S5.1, S5.2, S5.3, S8.2 (harness), continue S8.1 (→ 20 letters).
- **Expected demo (Day 14):** run the eval command; show a scorecard of extraction accuracy across 20 real letters; show a validator catching a hallucinated deadline.
- **Risks:** extraction accuracy plateaus below target (mitigate: per-type few-shot examples from golden set; contingency: narrow launch doc types from 8 to 5 — depth beats breadth); prompt iteration burns days (mitigate: eval harness makes each iteration a 2-minute measured experiment, which is exactly why it's built *this* sprint, not last).
- **DoD:** deadline exact-match ⩾85% on golden set for top-3 doc types; validators unit-tested; every field carries source spans.
- **Review checklist:** null-handling verified (no guessed fields); "other" doc type degrades gracefully; scorecard committed to repo.
- **Retro questions:** Which doc types are weakest and do they stay in launch scope? Is confidence scoring honest or decorative?

Sprint 3 (Days 15–21): "A human can use and verify it"

- **Business objective:** convert pipeline output into the user-facing experience that demos and screenshots are made of.
- **Engineering objective:** explanation engine, checklist, glossary, results page, source-highlight overlay.
- **Stories:** S6.1, S6.2, S6.3, S7.1, S7.2, S8.3 (integration/E2E), golden set → 30.
- **Expected demo (Day 21):** full user journey on a phone: photograph letter → results page → tap deadline → see it highlighted in the original scan → read explanation with glossary popovers.
- **Risks:** overlay coordinate math across scaled/rotated images (mitigate: normalize coordinates at OCR ingestion; test on multi-page early); explanation drifts into legal advice (mitigate: prompt constraint + output linter for advice phrases + disclaimer component).
- **DoD:** journey works on mobile; overlay accurate on all golden fixtures; explanation readability spot-checked with 2 non-native speakers.
- **Review checklist:** loading/error/empty states all designed; urgent-deadline flagging correct across timezones; screenshots taken (they're needed for Day 27, not Day 29).
- **Retro questions:** Would I trust this with my own Bescheid? What did the 2 test users not understand?

Sprint 4 (Days 22–30): "Ship, harden, and tell the story"

- **Business objective:** a live, private-by-design product plus the portfolio narrative that makes the month count.
- **Engineering objective:** GDPR baseline, deletion/purge, production hardening, full eval run, docs, demo video, launch.
- **Stories:** S7.3, S9.1, S9.2, S8.2 final run, S10.1, S10.2; bug-fix buffer.
- **Expected demo (Day 30):** public URL + repo + 3-minute video + README with published eval scorecard and failure analysis.
- **Risks:** buffer consumed by earlier slippage (contingency ladder, cut in this order: glossary popovers → 8th/7th doc types → analytics polish; **never cut**: validators, eval suite, deletion, disclaimer); demo-video perfectionism (mitigate: script it Day 27, one-take-plus-one policy).
- **DoD:** launch checklist (below) fully green.
- **Review checklist:** clean-machine setup test of README instructions; deletion actually deletes (verify at storage layer); Sentry catching a forced error.
- **Retro questions:** What would Sprint 5 be? What's the one metric to watch post-launch?

Phase 5 — Daily Development Checklist

Rhythm every working day: 10-min plan → deep work → tests before "done" → commit(s) → 5-min log note (these notes become LinkedIn posts and the retro). Weekends: Days 6–7 and 13–14 are working days in this plan; Days 20–21 light; use spare weekend hours only for golden-letter collection and writing, never for new features.

Day 1 · Obj: repo + environments. Tasks: scaffold Next.js + FastAPI monorepo, lint/format hooks, GitHub Actions CI, EU-region deploy target (e.g., Hetzner/Fly EU or Vercel fra1 + EU backend), Sentry EU. Output: CI-green deploys on push. Files: repo scaffold, `ci.yml`, `README.md` stub. Test: pipeline runs. **Commit:** `chore: project foundation + EU deploy pipeline`

Day 2 · Obj: walking skeleton. Tasks: upload endpoint (stub), job status polling, stub result render; landing page copy v0. Output: file → stub result on live URL. Files: `api/upload.py`, `web/app/page.tsx`, `web/app/result/`. Test: E2E happy-path smoke. **Commit:** `feat: end-to-end walking skeleton`

Day 3 · Obj: real upload + OCR bake-off. Tasks: multi-file/PDF intake, page splitting, size/type validation; run 5 fixture letters through 2–3 OCR options (e.g., Azure Document Intelligence vs Google Vision vs Tesseract baseline); compare text + bbox quality and EU-processing terms; **decide and record an ADR**. Output: signed OCR decision. Files: `ingestion/`, `docs/adr/001-ocr.md`. Test: rejection paths. **Commit:** `feat: document ingestion + ADR-001 OCR selection`

Day 4 · Obj: preprocessing. Tasks: deskew, contrast, downscale via OpenCV/Pillow; before/after OCR-confidence comparison on photo fixtures. Output: measurable OCR lift. Files: `ingestion/preprocess.py` + tests. Test: unit tests on transforms; fixture comparison script. **Commit:** `feat: image preprocessing with measured OCR improvement`

Day 5 · Obj: OCR integration. Tasks: OCR client wrapper, normalized `{text, page, bbox, confidence}` schema, multi-page merge, retries/timeouts. Output: real letters → structured OCR JSON. Files: `ocr/client.py`, `ocr/schema.py`, fixtures. Test: integration test on 3 fixtures. **Commit:** `feat: OCR pipeline with word-level coordinates`

Day 6 · Obj: quality gate + fixtures. Tasks: page-confidence threshold, "retake photo" UX with tips, tune on deliberately bad photos; expand golden set to 10 letters, start labeling. Output: bad input fails helpfully. Files: `ocr/quality.py`, `web/components/RetakePrompt.tsx`, `eval/golden/`. **Commit:** `feat: OCR quality gate + golden dataset v0`

Day 7 · Obj: Sprint-1 close. Tasks: bug sweep, phone-camera live test, sprint review against checklist, retro notes, plan Sprint 2; LinkedIn post #1 (OCR bake-off findings — concrete numbers, no hype). Output: Sprint-1 demo recorded (30-sec screen capture, raw). **Commit:** `chore: sprint 1 hardening + demo`

Day 8 · Obj: classification. Tasks: LLM classifier with few-shot per type, "other" fallback, eval against labeled golden letters. Output: ≥90% on current set. Files: `pipeline/classify.py`, `prompts/classify/`. Test: classifier eval script. **Commit:** `feat: document-type classification`

Day 9 · Obj: extraction schemas. Tasks: Pydantic schemas for top-4 types + generic; prompt scaffolding for JSON-constrained extraction; `{value, confidence, source_span}` wrapper. Files: `pipeline/schemas/`, `prompts/extract/`. Test: schema unit tests. **Commit:** `feat: typed extraction schemas`

Day 10 · Obj: extraction v1. Tasks: end-to-end extraction for top-4 types; source-span linking back to OCR words; null-not-guess policy enforced in prompt + parser. Output: real letter → structured fields with provenance. Files: `pipeline/extract.py`. Test: integration on fixtures. **Commit:** `feat: structured extraction with source provenance`

Day 11 · Obj: validator layer. Tasks: date parsing/ordering rules, deadline-vs-letter-date check, legal-reference whitelist check, amount checks; failures downgrade confidence + flag. Files: `pipeline/validate.py` + full unit coverage. **Commit:** `feat: deterministic validation layer (100% unit-tested)`

Day 12 · Obj: eval harness. Tasks: scoring script (per-field precision/recall, deadline exact-match), scorecard markdown output, fast-subset CI job; first full run; fix the two ugliest failure patterns. Output: committed scorecard v1. Files: `eval/run.py`, `eval/report.md`. **Commit:** `feat: automated eval harness + baseline scorecard`

Day 13 · Obj: accuracy iteration. Tasks: measured prompt iterations against harness; per-type few-shots from golden set; extend to remaining launch types or execute the narrow-

scope contingency (decision point — record it). Output: ≥85% deadline exact-match on top-3 types. **Commit:** `feat: extraction accuracy iteration (scorecard vN)`

Day 14 · Obj: Sprint-2 close. Tasks: confidence-tier logic (S5.3) wired; golden set → 20; sprint review, retro, Sprint-3 plan; LinkedIn post #2 (the eval harness — show the scorecard). **Commit:** `chore: sprint 2 close + confidence scoring`

Day 15 · Obj: explanation engine. Tasks: grounded explanation prompt (document + fields only, outside knowledge forbidden), length/readability constraints, advice-phrase linter, disclaimer component. Files: `pipeline/explain.py`, `prompts/explain/`, `web/components/Disclaimer.tsx`. Test: linter unit tests; groundedness spot-check protocol. **Commit:** `feat: grounded plain-language explanation`

Day 16 · Obj: checklist + glossary. Tasks: action-checklist derivation with urgency flags and empty state; 50-term glossary JSON + popover component. Files: `pipeline/actions.py`, `data/glossary.json`, `web/components/Glossary.tsx`. **Commit:** `feat: action checklist + Amtsdeutsch glossary`

Day 17 · Obj: results page. Tasks: summary card, explanation section, checklist, processing states with honest step labels, designed error/empty states. Files: `web/app/result/*`. Test: mobile viewport pass. **Commit:** `feat: results experience v1`

Day 18 · Obj: source-highlight overlay, part 1. Tasks: document viewer with page navigation; coordinate normalization; bbox overlay rendering at any scale. Files: `web/components/DocViewer.tsx`, `web/lib/coords.ts` + unit tests on the math. **Commit:** `feat: document viewer with coordinate-mapped overlays`

Day 19 · Obj: overlay part 2. Tasks: field↔highlight interaction (click field → scroll + highlight), multi-page handling, low-confidence "verify against source" prompts. Output: the signature demo moment works. Test: all golden fixtures visually verified. **Commit:** `feat: click-to-verify source highlighting`

Day 20 · Obj: test hardening. Tasks: Playwright E2E happy path; integration tests OCR→extract→explain on 3 fixtures; fix what they catch. Files: `e2e/`, `tests/integration/`. **Commit:** `test: E2E + pipeline integration coverage`

Day 21 · Obj: Sprint-3 close + usability. Tasks: 2 non-native speakers run the real journey on their phones, notes taken; top-3 friction fixes; screenshots captured properly (clean data, both themes); sprint review/retro; LinkedIn post #3 (the highlight feature, 20-sec clip). **Commit:** `chore: sprint 3 close + usability fixes`

Day 22 · Obj: privacy features. Tasks: no-account flow confirmed; one-click delete (files + DB, verified at storage layer); 24h auto-purge job; API no-training/retention flags set and documented. Files: `api/delete.py`, `jobs/purge.py` + tests. **Commit:** `feat: deletion + auto-purge (GDPR baseline)`

Day 23 · Obj: privacy page + landing. Tasks: plain-language privacy page matching actual behavior; landing page final copy (problem, how it works, trust section); disclaimer review

everywhere. Files: `web/app/privacy/`, `web/app/page.tsx`. **Commit:** `feat: privacy page + launch landing`

Day 24 · Obj: production hardening. Tasks: rate limiting, file-size abuse guards, structured logging, uptime monitor, cost guardrails on LLM/OCR calls; load-test 10 concurrent analyses. **Commit:** `chore: production hardening`

Day 25 · Obj: final eval + failure analysis. Tasks: full eval on all 30 golden letters; write the **failure-analysis section** (what fails, why, what would fix it); freeze the scorecard for README. Files: `eval/report.md`, `docs/failure-analysis.md`. **Commit:** `docs: final eval scorecard + failure analysis`

Day 26 · Obj: README + architecture. Tasks: architecture diagram (Mermaid/Excalidraw), README rewrite (problem → demo GIF → architecture → eval results → failure analysis → competitor acknowledgment → setup); ADR index. **Commit:** `docs: portfolio-grade README + architecture`

Day 27 · Obj: demo assets. Tasks: script the 3-minute video (journey + verify-in-source moment + deletion), record (one take + one retry), edit lightly, host; finalize screenshot set. **Commit:** `docs: demo video + screenshots`

Day 28 · Obj: clean-machine test + bug sweep. Tasks: follow README setup on a fresh environment, fix every gap; triage remaining bugs (fix P0s, log the rest as issues — open issues signal honesty, not failure). **Commit:** `fix: launch bug sweep + verified setup docs`

Day 29 · Obj: launch checklist. Tasks: run the full launch checklist (below); DNS/domain; final Sentry + monitor verification; soft-launch to 5 friendly users, watch error tracking. **Commit:** `chore: launch`

Day 30 · Obj: publish + retrospective. Tasks: LinkedIn launch post (demo video + eval numbers + one honest lesson — specifics, not "excited to announce"); pin repo; write the project retrospective in `docs/retro.md` (this doc is *also* portfolio material); define post-MVP Sprint-5 candidates from the backlog. **Commit:** `docs: launch retrospective`

Phase 6 — Dependencies & Critical Path

Critical path: S1.1 → S1.2 → S2.1 → S3.1 → S5.1 → S5.2 → S7.2 → S10.2. Slippage anywhere on this line moves launch. Protect it: critical-path work gets morning deep-work hours; everything else gets afternoons.

Parallelizable: golden-set collection/labeling (evenings, ongoing); glossary curation; landing/privacy copy; LinkedIn drafts — none of these ever justify touching critical-path time.

Blocks the most work: S3.1 (OCR with bboxes). It feeds classification, extraction, and the overlay. That is why it's a week-1 deliverable and why the bake-off is timeboxed to one day.

Architecture decisions that must not change later (each gets an ADR in week 1–2):

1. OCR provider and the normalized coordinate schema — the overlay math and all fixtures depend on it.
2. Per-type Pydantic schemas with `{value, confidence, source_span}` field wrappers — the contract between pipeline and frontend.
3. EU-region hosting and no-account/session model — retrofitting privacy architecture is a rewrite.
4. Eval harness format — golden fixtures are an investment; churning their schema burns it.
5. Language as an output parameter (even while shipping EN-only) — makes post-MVP languages config, not surgery.

Phase 7 — Risk Register

#	Risk	Likelihood	Impact	Mitigation	Contingency
R1	**Scope creep** (self-inflicted; the prompt-driven urge to add reply drafting "real quick")	High	High	Backlog rule; sprint goals written down; this document	Any added feature must name the feature it replaces
R2	OCR quality on phone photos below usable	Medium	High	Day-3 bake-off with real fixtures; preprocessing; quality gate	Restrict MVP input to PDFs + flatbed scans; state it honestly
R3	LLM hallucination (invented deadlines/amounts)	High	**Critical** (user misses a visa/objection deadline)	Validator layer; source-span provenance; confidence flags; null-not-guess; eval harness	Any field failing validation renders as "couldn't reliably extract — check highlighted area" rather than a value
R4	Extraction accuracy plateaus <85%	Medium	High	Measured prompt iteration via harness; per-type few-shots	Cut launch doc types 8→5; depth over breadth
R5	GDPR gap (retention claim ≠	Low	High	Deletion verified at	Delay launch before shipping a

#	Risk	Likelihood	Impact	Mitigation	Contingency
	implementation)			storage layer; auto-purge tested; privacy page written *from* the code	false privacy claim — non-negotiable
R6	Crossing into legal advice (RDG)	Medium	High	Explanation-only framing; advice-phrase linter; disclaimer; reply drafting deferred until legal review	Remove any "you should file X" phrasing; point to Beratungsstellen
R7	Prompt regressions from model/prompt changes	High	Medium	Eval on CI subset; prompts versioned in repo; scorecard per change	Pin model versions; revert policy
R8	Solo-dev burnout / pace slippage	Medium	High	Realistic 6–8h days; buffer in Sprint 4; retro checkpoints	Execute the cut ladder (glossary → doc types → polish); never cut tests/eval/privacy
R9	Deployment/provider failure near launch	Low	Medium	Deployed from Day 1, so launch is a non-event	Second region/provider documented in ADR
R10	Cost blowout on LLM/OCR calls	Low	Medium	Per-analysis cost logged; rate limiting; caching of OCR results	Cap free analyses per session

Phase 8 — Testing Strategy

Layer	What	Gate
Unit	Validators (100% coverage — they're the safety net), coordinate math, preprocessing transforms, checklist derivation, advice-phrase linter	Feature not "done" without them
Integration	OCR→classify→extract→explain on 3 canonical fixtures (photo, scanned PDF, multi-page)	CI on every PR
Eval (the star)	Full golden-set scoring per field per doc type; fast subset in CI; full run at sprint boundaries	Scorecard must not regress between sprints
E2E	Playwright: upload → results → highlight interaction → delete	CI nightly + pre-launch
Manual	Real phone camera test each sprint; 2-user usability session Day 21; clean-machine setup Day 28	Sprint review checklists
Edge cases	Handwritten additions, rotated pages, coffee-stained scans, letters with no deadline, letters in the "other" class, 10-page documents, 20MB uploads, duplicate uploads	Fixture-backed where feasible; else documented in failure analysis
Failure cases	OCR provider timeout, LLM malformed JSON (parser retry then graceful degrade), validator rejection path, deletion mid-processing	Integration-tested
Performance	10 concurrent analyses; P95 end-to-end < 60s with honest progress UI	Day 24 load test

Per-feature completion rule: a feature is complete when its ACs pass, its tests exist in CI, its failure mode is designed (not default-error), and it works on a phone.

Phase 9 — GitHub Project Board (Kanban)

Columns: **Backlog · Sprint 1 · Sprint 2 · Sprint 3 · Sprint 4 · In Review · Testing · Done**

- **Backlog:** reply drafting (+RDG legal review task), case threads, deadline reminders, accounts/history, languages 2–5, Widerspruch flow, native apps, B2B white-label, family sharing, timeline view, insurance/gov partnership exploration
- **Sprint 1:** S1.1, S1.2, S2.1, S2.2, S3.1, S3.2, golden-letters-batch-1, ADR-001-OCR, LinkedIn-post-1

- **Sprint 2:** S4.1, S5.1, S5.2, S5.3, S8.2, golden-letters-batch-2, accuracy-iteration, ADR-002-schemas, LinkedIn-post-2
- **Sprint 3:** S6.1, S6.2, S6.3, S7.1, S7.2, S8.3, usability-session, screenshots, LinkedIn-post-3
- **Sprint 4:** S7.3, S9.1, S9.2, final-eval + failure-analysis, S10.1, S10.2, hardening, clean-machine-test, launch-checklist, LinkedIn-launch-post, retro-doc
- **In Review / Testing:** transient columns; PRs sit in Review, features awaiting fixture/E2E verification sit in Testing
- **Done:** everything above, eventually — use GitHub Milestones `Sprint 1–4` + `Launch` so the burndown is visible to repo visitors

---

Phase 10 — Sprint Deliverables (what must exist)

End of	Must exist
Sprint 1	Live URL; working upload (phone + PDF); OCR JSON with bboxes; quality gate; 10 labeled golden letters; ADR-001; raw 30-sec demo clip
Sprint 2	Classification (≥90%); schema-validated extraction with provenance + confidence; validator layer fully unit-tested; eval harness + committed scorecard; 20 golden letters
Sprint 3	Complete user journey on mobile; results page; click-to-verify source highlighting; explanation + checklist + glossary; E2E tests; screenshots; usability findings
Sprint 4 / Launch	Deletion + auto-purge verified; privacy page; hardened prod deploy; final scorecard + failure analysis; architecture diagram; portfolio README; 3-min demo video; launch post; retrospective doc

---

Phase 11 — Portfolio Readiness

Commits: conventional-commit style (`feat:`, `fix:`, `test:`, `docs:`, `chore:`), each a coherent unit with a one-line "why" in the body when non-obvious. No `wip`, no `final final v2`. Hiring managers *do* read the commit log — it's the most honest artifact in the repo.

Cadence of proof:

- Day 7: raw 30-sec clip (OCR over real letter) — imperfect on purpose, it shows process
- Day 14: eval scorecard screenshot — the "this person measures quality" moment
- Day 21: 20-sec highlight-feature clip + full screenshot set

- Day 27: final 3-minute demo video, scripted: problem (20s) → journey (90s) → verify-in-source moment (30s) → eval numbers + honest limitation (30s) → deletion (10s)

Documentation timing: ADRs the day decisions are made (not reconstructed later — reconstructed ADRs read fake); README stub Day 1, real rewrite Day 26; failure analysis Day 25 while the eval run is fresh; `docs/retro.md` Day 30.

LinkedIn cadence (4 posts, matching your daily-posting motion): Day 7 (OCR bake-off with numbers), Day 14 (why I built an eval suite before more features), Day 21 (the source-highlighting feature and why trust beats fluency), Day 30 (launch: video, scorecard, one honest lesson, link). Format each as a specific engineering decision + evidence + takeaway. Skip "excited to announce" and "game-changer" — the audience you're targeting (EMs, senior PMs) filters those out reflexively. Comments from German tech folks on these posts are worth more than the impressions.

Engineering-maturity signals, in priority order: published eval results → deterministic validation layer → honest failure analysis → provenance/source-highlighting → ADRs → test coverage on the seams → verified GDPR behavior → clean commit history. Notice that "which LLM you used" is not on the list.

Final Launch Checklist (Day 29)

- [ ] Full golden-set eval run; scorecard in README matches latest run
- [ ] E2E suite green on production URL
- [ ] Phone-camera journey verified on iOS and Android browsers
- [ ] Highlight overlay verified on multi-page fixture
- [ ] "Delete everything" verified at storage layer; 24h purge job confirmed ran
- [ ] Privacy page claims audited against implementation
- [ ] Disclaimer present on every results view; advice-linter passing
- [ ] Rate limits + cost caps active; per-analysis cost known
- [ ] Sentry receiving events (test error fired); uptime monitor live
- [ ] README setup verified on clean machine; demo video linked and playing
- [ ] ADRs 001–005 committed; open issues triaged and labeled
- [ ] Launch post drafted with video + scorecard + repo link
- [ ] 5 soft-launch users completed the journey without hand-holding